

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Lecture 11

- Normal (or Gaussian) Distribution
- Kernel Density Estimation (KDE)

Class participation question

Post your answers to the question below in Slack before the next class. Please prefix your post with: “Binomial 2/23”

- **Questions:**

The number of side effects in a clinical trial out of 100 patients can be modeled using a binomial distribution.

1. Explain how this scenario satisfies the conditions required for a binomial model: fixed number of trials, independent trials, binary outcomes, and constant probability of success
2. What real-life questions could be answered with this model?

$$P(X \geq 1) = 1 - P(X = 0) \geq 0.8$$

*n = 100
p = known
m = 0*

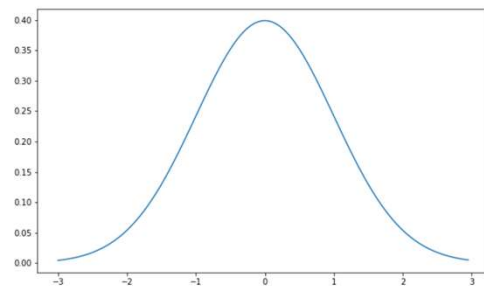
The normal distribution

- The normal or Gaussian distribution is a continuous probability distribution.
- Numerous real-world phenomena can be modeled or approximated using normal distributions.
- Some examples:
 - The heights of males in a population
 - Measurement errors from an instrument
 - Duration of cab rides
- Many statistical tests assume that your data is distributed according to a normal distribution.
 - More on this later.



Properties of the normal distribution

- The normal distribution is a continuous distribution.
 - Typically, graphically represented as a continuous curve.
 - As opposed to the discrete distribution, the continuous can usually take any values either in a closed or open range.
 - The probability space is uncountably infinite.
- The normal distribution is bell-shaped.
 - Symmetric with a mean in middle.



Properties of the normal distribution (cont'd.)

- The parameters for a normal distribution are the location of its center (mean) and spread (standard deviation).
 - Mean (μ) governs where the middle of the distribution lies on the x-axis.
 - Standard deviation (σ) determines how wide the bell is and can also be referred to as the *scale*.
- To describe that a random variable is distributed according to a normal distribution with parameters μ and σ , we write: $X \sim \mathcal{N}(\mu, \sigma)$
- As with the binomial, two normal distributions with different parameters tell different stories:

$$X \sim \mathcal{N}(0, 1) \neq Y \sim \mathcal{N}(1, 1) \neq Z \sim \mathcal{N}(2, 2)$$

```

from scipy.stats import norm

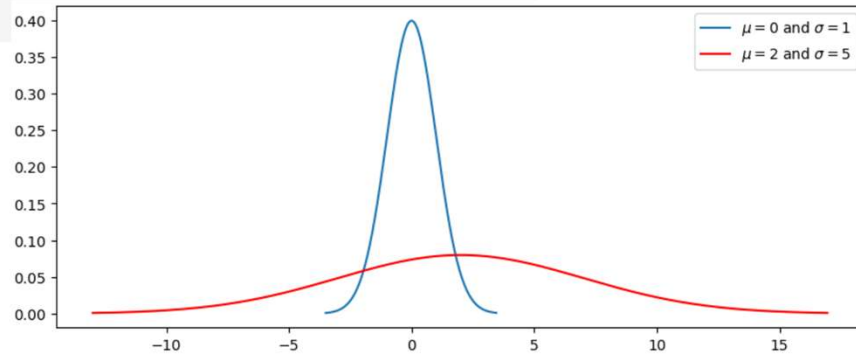
plt.figure(figsize=(10, 4))

x = np.arange(-3.5, 3.5, 0.05)
p_x = [norm.pdf(val, 0, 1) for val in x]
plt.plot(x, p_x, label="$\\mu = 0$ and $\\sigma = 1$")

x = np.arange(-13, 17, 0.05)
p_x = [norm.pdf(val, 2, 5) for val in x]
plt.plot(x, p_x, label="$\\mu = 2$ and $\\sigma = 5$", color='r')

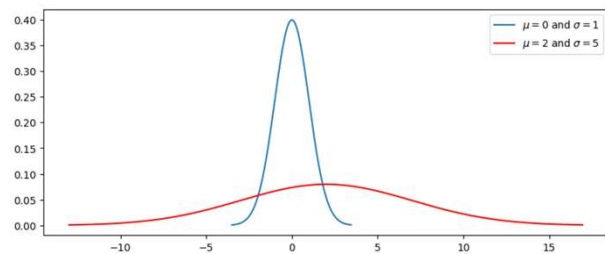
plt.legend();

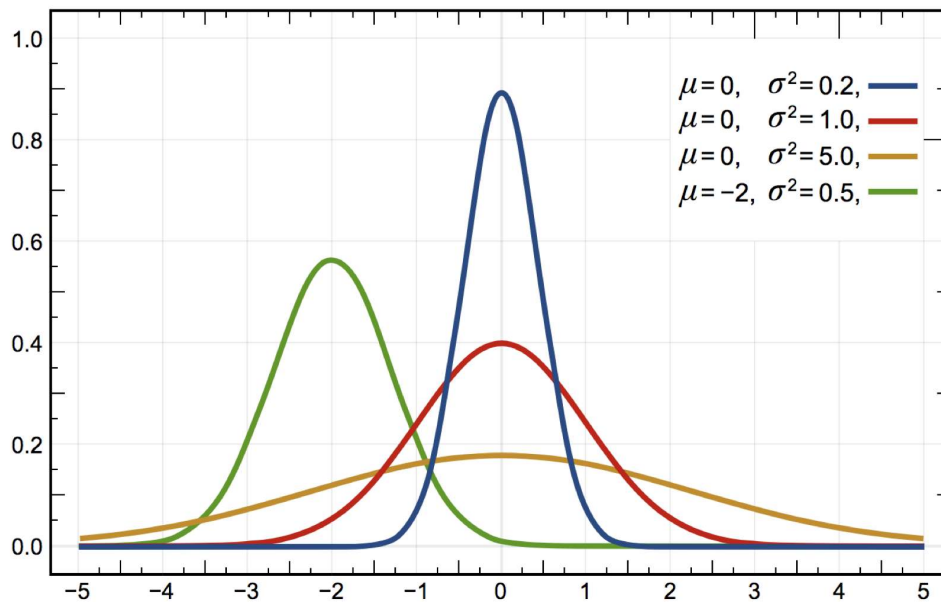
```



The spread and density

- Blue curve ($\mu = 0$ and $\sigma = 1$): Small standard deviation $\rightarrow$ density tightly concentrated around the mean; extreme values are unlikely.
- Red curve ($\mu = 2$ and $\sigma = 5$): Larger standard deviation $\rightarrow$ density more spread out; extreme values are more likely and peak height is lower.
- Think of this in terms of the probability distribution of the biased die from the binomial example.
 - If you were to repeatedly draw a random value – just like when we were sampling from the die – what do you think the values obtained from the red distribution would look like?

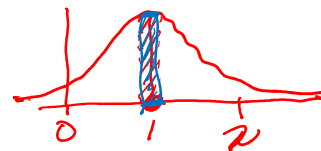




Probability density function for a normal distribution

- The probability density function (pdf) for a random variable $X \sim \mathcal{N}(\mu, \sigma)$ is:

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



- Since the sample space is uncountably infinite, the probability of a single outcome is 0.
- Therefore, instead of computing the pmf, continuous random variables compute the pdf.

```
# We can manually implement the PDF
```

```
def computePDF_normal (x, mu, sigma):  
    return 1.0/(sigma * np.sqrt(2 * np.pi)) * np.exp(-(x - mu)**2.0 / (2 * sigma**2))
```

```
print(computePDF_normal(220, 174, 11))
```

```
5.7827254290572115e-06
```

```
from scipy.stats import norm
```

```
print(norm.pdf(220, 174, 11))
```

```
5.7827254290572225e-06
```

std
mean

```
[float(norm.rvs(174, 11)) for _ in range(5)]
```

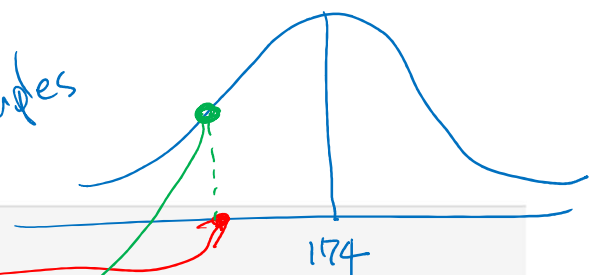
```
[168.77066332043364,  
163.32754716678167,  
174.38162516911424,  
187.81420293802972,  
176.192046271956]
```

```
x = norm.rvs(174, 11, 5)  
x
```

```
array([168.18540865, 163.78559585, 174.25226483, 176.83390004,  
176.91144946])
```

```
norm.pdf(x, 174, 11)
```

```
array([0.03153862, 0.02356562, 0.03625794, 0.03508366, 0.03501913])
```

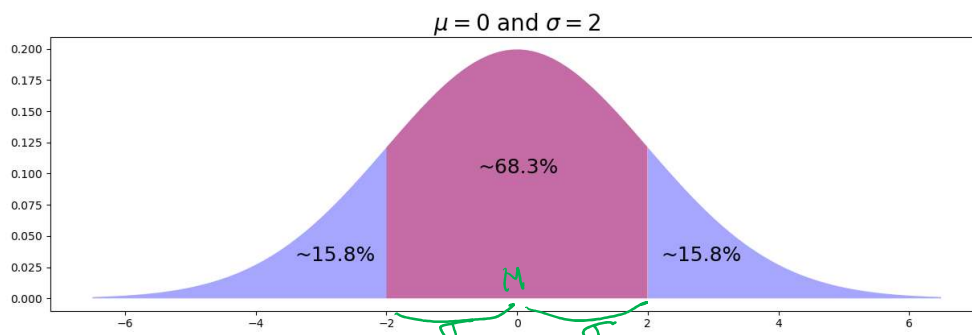


pdf versus pmf

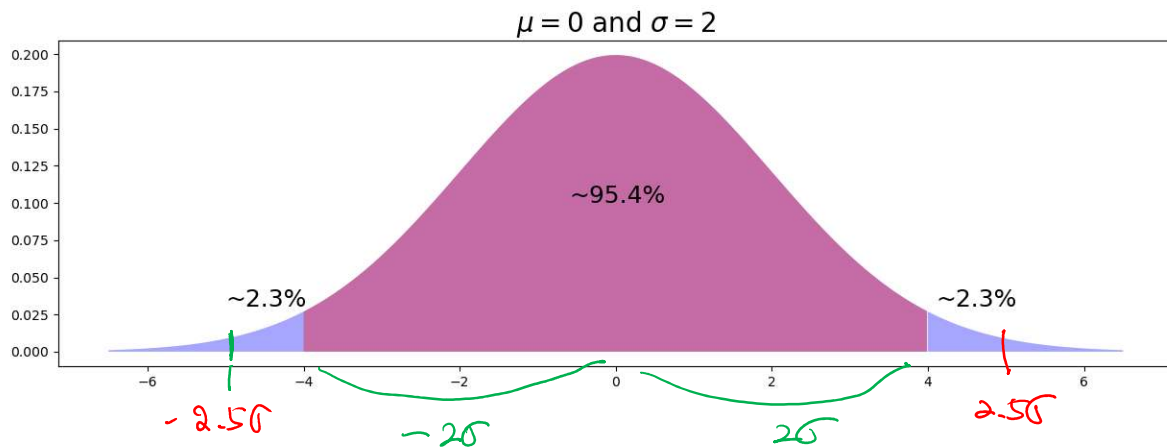
- A pdf is conceptually similar to a pmf, but instead of giving you a probability, it tells you the density around a particular point.
 - You can think of it as probability mass spread continuously along the real line, where the height of the curve indicates how likely values are and the **area under the curve** gives actual probabilities.
 - E.g., in the $X \sim \mathcal{N}(0, 1)$, probability density around 0 is much higher than the density around -2.5 .
- Since X is continuous, the pdf is also continuous.
- The total area under the pdf curve sums to 1.

Data in the distribution – 1σ

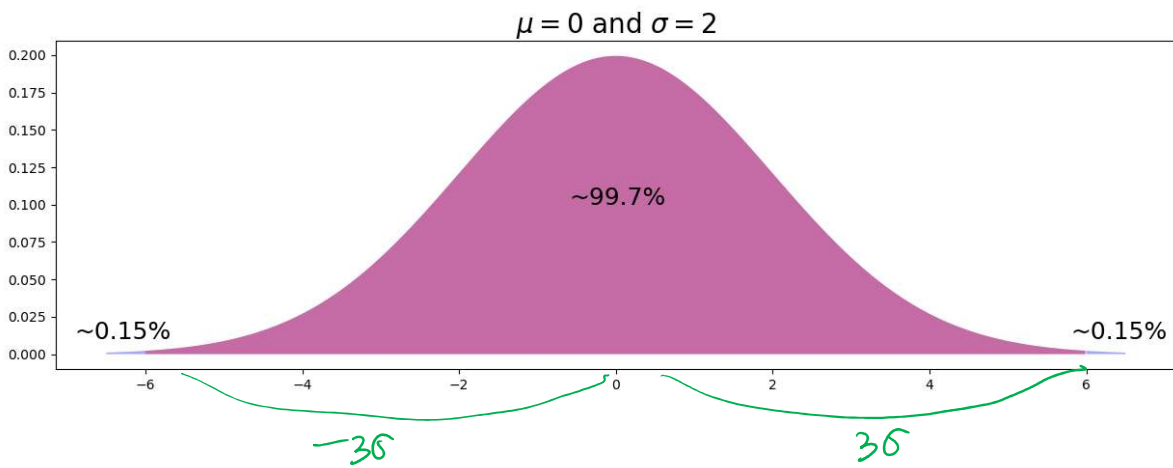
- In the normal distribution, the area with the highest density lies close to the mean.
 - Thus, when we sample from a normal distribution, the bulk of the observations generated lie near the mean.



Data in the distribution – 2σ



Data in the distribution – 3σ



Questions

$$\text{Scores} \sim N(70, 10)$$

- Suppose test scores are approximately normal with mean 70 and standard deviation 10.
 - a) Roughly what percentage of students do you expect to score between 60 and 80?
 - b) Is a score of 95 unusual? Why or why not?

$$\begin{array}{ccc} & \approx 68\% & \\ 60 \leq \text{scores} \leq 80 & & \\ \parallel & & \parallel \\ 70 - 10 & & 70 + 10 \\ \mu - \sigma & & \mu + \sigma \end{array}$$
$$95 = 70 + 25$$
$$= \mu + 2.5\sigma$$

Computing the pdf in Python

- We can easily compute the pdf in Python using scipy.

```
import scipy.stats
norm_0_1 = scipy.stats.norm(0, 1)
print(norm_0_1.pdf(2))
```

- pdf can also take a range of values: `print(norm_0_1.pdf([-2, -1, 0, 1, 2]))`
- `scipy.stats.norm` creates a frozen distribution.
 - This can be thought of a constructor in object-oriented programming.
 - Can also invoke pdf using static methods:
`scipy.stats.norm.pdf(0, loc=0, scale=1)`
- Which of the pdf values is the largest and why?
- Which of the pdfs is larger, `pdf(-2)` or `pdf(2)`?

```

norm_0_1 = scipy.stats.norm(0, 1)
print(norm_0_1.pdf(2))

0.05399096651318806

print(scipy.stats.norm.pdf(2, 0, 1))

0.05399096651318806

print(norm_0_1.pdf([-2, -1, 0, 1, 2]))

[0.05399097 0.24197072 0.39894228 0.24197072 0.05399097]

print(scipy.stats.norm.pdf([-2, -1, 0, 1, 2], 0, 1))

[0.05399097 0.24197072 0.39894228 0.24197072 0.05399097]

# return 3 random variates from Normal with mean 0 and std = 1

print(norm_0_1.rvs(3))
print(scipy.stats.norm.rvs(loc=0, scale=1, size=3))

[ 1.14651253 -1.08817329 -1.19209425]
[ 0.62267766 -0.13288105 -0.57306768]

```

The sum of PDF $\neq 1$

```

print(sum(norm_0_1.pdf([-2, -1, -0.5, 0, 0.5, 1, 2])))

1.6949963159946946  $\neq 1$ 

```

- Why is the sum of the pdfs for the small subsample of the sample space greater than 1?

```

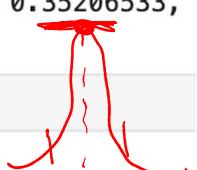
norm_0_1.pdf([-2, -1, -0.5, 0, 0.5, 1, 2])

array([0.05399097, 0.24197072, 0.35206533, 0.39894228, 0.35206533,
       0.24197072, 0.05399097])

print(scipy.stats.norm.pdf(1, loc=1, scale=0.1)) →
3.989422804014327

```

$\mu=1$ $\sigma=0.1$



Illustrating density

- To illustrate the idea that dense regions contribute more random samples, we sample 1,000 values from a normal distribution $\mathcal{N}(\mu = 0, \sigma = 2)$.
- We will plot the values as a histogram.
 - We bin the values into 20 bins and plot them as a histogram.
 - Counts for each bin are shown on the y-axis.

```
np.random.seed(225)
gaussianSamples = np.random.normal(0, 2, 1000)
gaussianSamples[0:10]

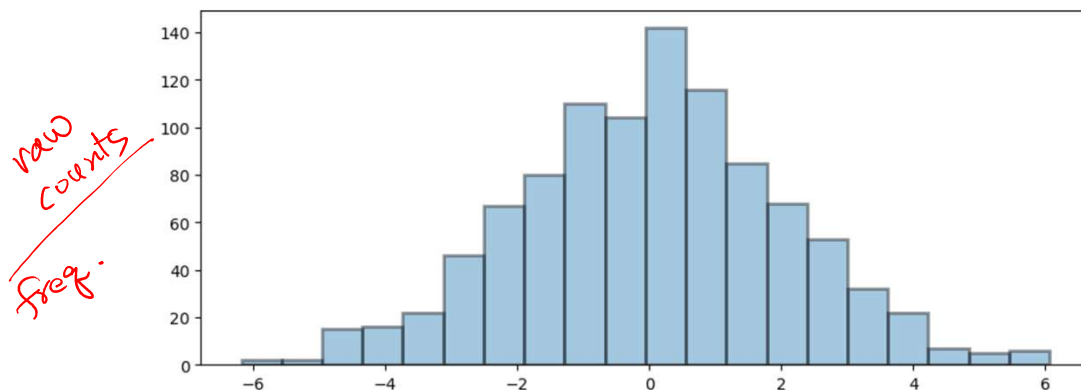
array([ 0.62002045, -3.6654824, -1.21798399, -2.60499913,  1.77670804,
        0.41631649, -2.81801219,  3.74434068,  0.68670599, -0.72788997])

len(gaussianSamples)

1000
```

```
# plot the counts rather than the density
```

```
plt.figure(figsize=(10, 4))
counts, _, _ = plt.hist(gaussianSamples, bins=20, edgecolor="k", linewidth=2, alpha=0.4)
```



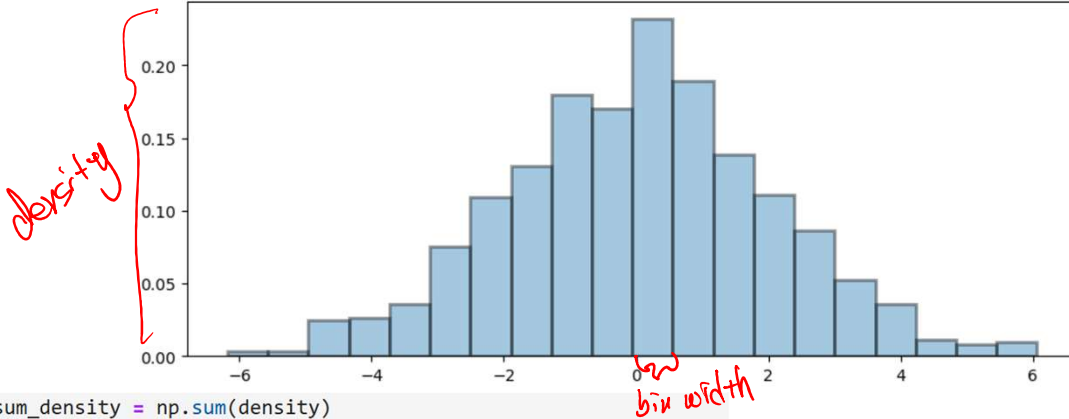
```
print("Total count =", np.sum(counts))
```

```
Total count = 1000.0
```

```
# plots the density rather than the counts
```

```
plt.figure(figsize=(10, 4))
```

```
density, bins, _ = plt.hist(gaussianSamples, bins=20, edgecolor="k", linewidth=2, alpha=0.4, density=True)
```



```
sum_density = np.sum(density)
```

```
print("Sum of densities =", sum_density)
```

```
print("Area under the curve =", sum_density * np.diff(bins)[0])
```

```
Sum of densities = 1.6333097581920162
```

```
Area under the curve = 1.0
```

Comparing the density curve and the histogram

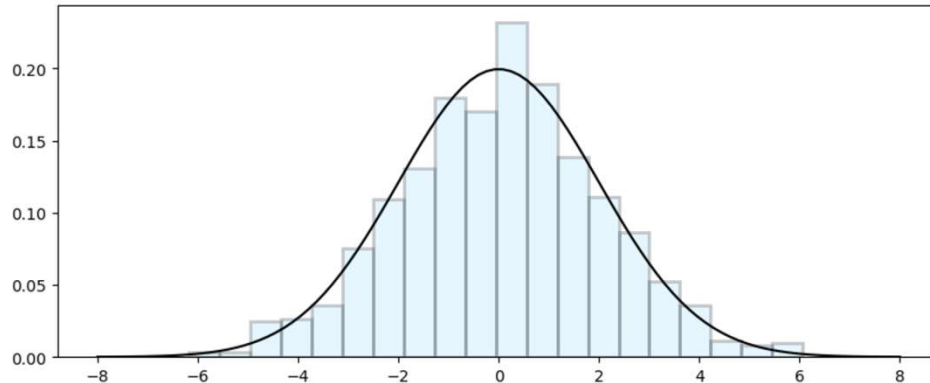
- We plot the pdf over the histogram to see if it matches the sequence data.
- Should the histogram match the density curve exactly?

```
plt.figure(figsize=(10, 4))

normal_0_2 = scipy.stats.norm(0, 2)
x_axis = np.linspace(-8, 8, 100)
y_axis = normal_0_2.pdf(x_axis)

plt.plot(x_axis, y_axis, color="k")

# Note that the param density=True is necessary here
_ = plt.hist(gaussianSamples, bins=20, edgecolor="k", color='#80d4ff', linewidth=2, alpha=0.2, density=True)
```



```
some_vals = [-2, -1, -0.5, -0.2, -0.1, 0, 0.1, 0.2, 0.5, 1, 2]

mu = 0; sigma = 2
print(f"For mean = {mu} and std = {sigma}:")

for val in some_vals:
    print(f"\tThe probability density of x = {val: >4} is {norm.pdf(val, 0, 2):.7f}")
```

For mean = 0 and std = 2:

```
The probability density of x = -2 is 0.1209854
The probability density of x = -1 is 0.1760327
The probability density of x = -0.5 is 0.1933341
The probability density of x = -0.2 is 0.1984763
The probability density of x = 0 is 0.1994711
The probability density of x = 1 is 0.1760327
The probability density of x = 0 is 0.1994711
The probability density of x = 0.1 is 0.1992220
The probability density of x = 0.2 is 0.1984763
The probability density of x = 0.5 is 0.1933341
The probability density of x = 1 is 0.1760327
The probability density of x = 2 is 0.1209854
```

```
print("The sum of the probability densities computed above is: ", end="")
print(sum(norm.pdf(some_vals, 0, 2)))
```

The sum of the probability densities computed above is: 2.1518536163942685

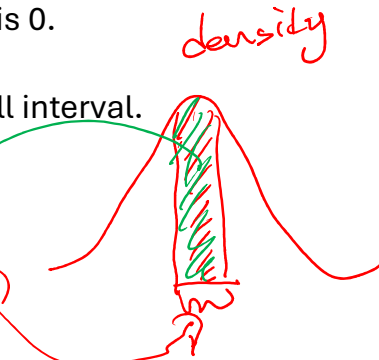
The sum of pdf values

- Why is the sum greater than 1? Aren't the probabilities of the elements in the sample space supposed to sum to 1?
- Remember that the probability at a single point is 0.
 - The sample space is infinite.
- Therefore, pdf computes the probability of small interval.
 - E.g., $x - 0.01 < x < x + 0.01$
- We know that:

$$\text{density} = \frac{\text{mass}}{\text{interval width}}$$

- So, we can convert back to mass such that

$$\text{mass} = \text{density} \times \text{interval width}$$



$\sigma=2$

```
norm_cdf_sum = 0
for x in np.arange(-8, 8, 0.0001):
    norm_cdf_sum += computePDF_normal(x, 0, 2) * 0.0001
print("The sum of the probability densities computed above is:", norm_cdf_sum)

# Use of the cumulative distribution function (cdf)
print("The sum of the probability densities computed using cdf:", norm.cdf(8, 0, 2))
```

The sum of the probability densities computed above is: 0.9999366575184352
The sum of the probability densities computed using cdf: 0.9999683287581669

≈ 1

Kernel Density Estimation (KDE)

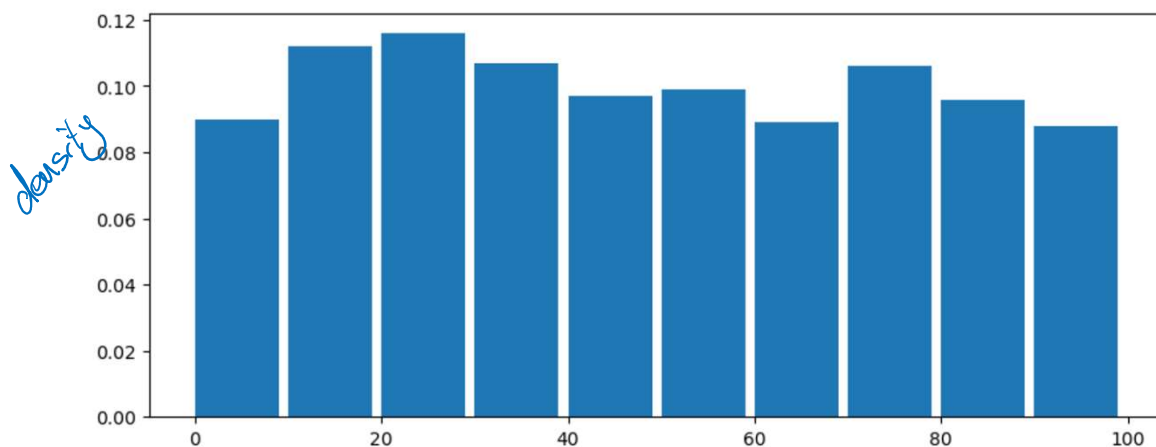
Introduction to kernel density estimation: Histograms

- The histogram of random draws (e.g., `randint(0, 100)`) can be generated by tallying the number of data points within each bin of size 10.
- The bins are: 0-9, 10-19, 20-29, ...
- Rather than code the process using Python standard library functionality, we will use numpy to generate the counts.

Frequencies and probability distributions

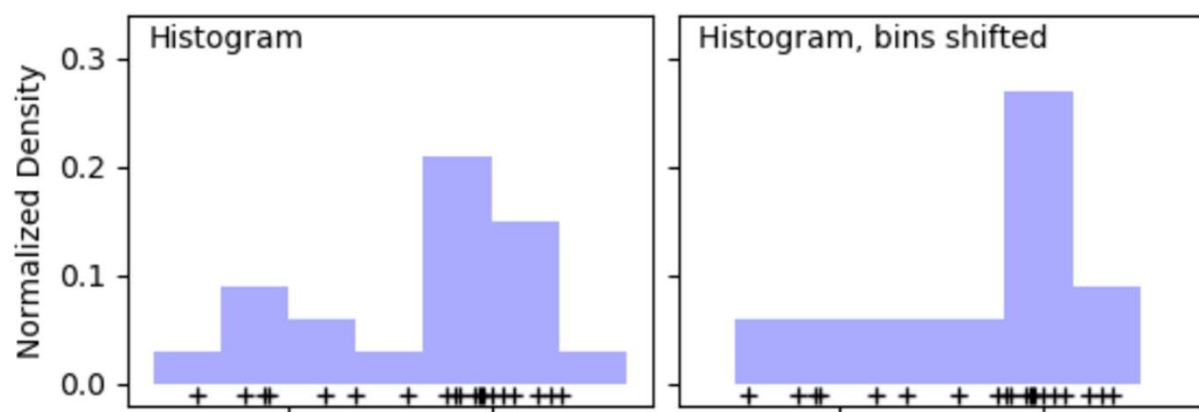
- Recall that this is not a valid probability distribution according to the definition of probability distribution.
 - Why?
 - How can we turn it into a probability distribution?

```
normalized_counts = counts_and_bins[0] / sum(counts_and_bins[0])  
  
plt.figure(figsize=(10, 4))  
plt.bar(counts_and_bins[1][:-1], normalized_counts, width=9, align='edge')  
plt.show()
```



Nonparametric density estimation for discrete random variables

- Back to histograms: `np.histogram` allows computing normalized counts (pmf style).
 - `np.histogram(numbers, bins, density=True)`
- A significant problem with histograms is that the choice of bin boundaries can have a disproportionate effect on the resulting visualization.
- Values for a bin can change based on the start and end location of the bin.
 - The probability distribution and, therefore, the interpretation of the data can become inconsistent when using two different bin boundaries.



Histograms on same data: bins are of the same size, but histogram on the right has bins shifted to the right.

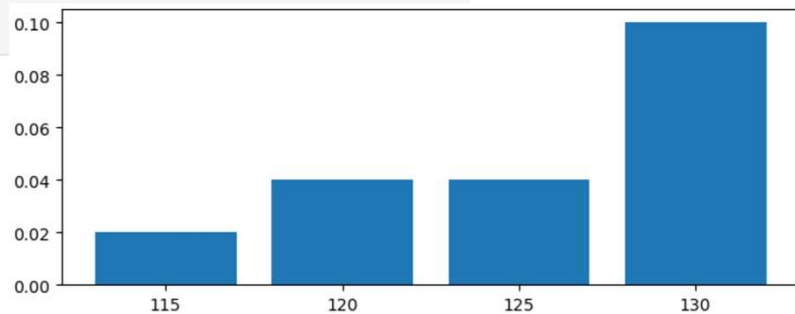
```
data = [119, 120, 121, 125, 129, 130, 130.1, 130.5, 130.7, 131]
bins_2 = range(115, 136, 5)
print(list(bins_2))
```

```
[115, 120, 125, 130, 135]
```

```
counts = np.histogram(data, bins_2, density=True)
print(counts[0])
```

```
plt.figure(figsize=(8, 3))
plt.bar(x=counts[1][:-1], height=counts[0], width=4)
plt.xticks(bins_2[:-1])
plt.show()
```

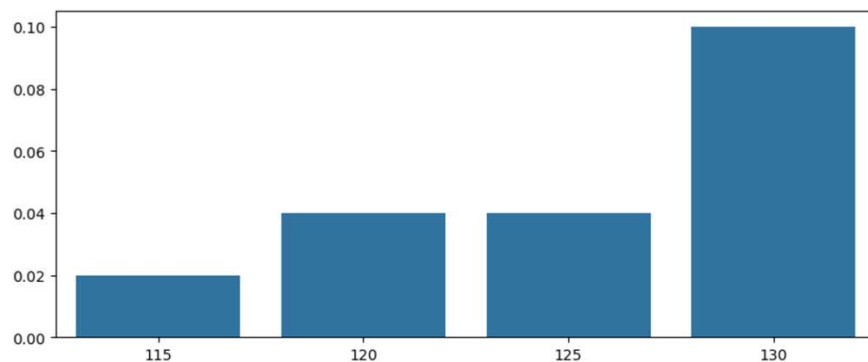
```
[0.02 0.04 0.04 0.1 ]
```



```
# Plotting using Seaborn
```

```
import seaborn as sns
```

```
plt.figure(figsize=(8, 3))
sns.barplot(x=counts[1][:-1], y=counts[0])
plt.show()
```



```
data = [119, 120, 121, 125, 129, 130, 130.1, 130.5, 130.7, 131]
```

```
bins_3 = range(114, 136, 5)
```

```
print(list(bins_3))
```

```
counts = np.histogram(data, bins_3, density=True)
```

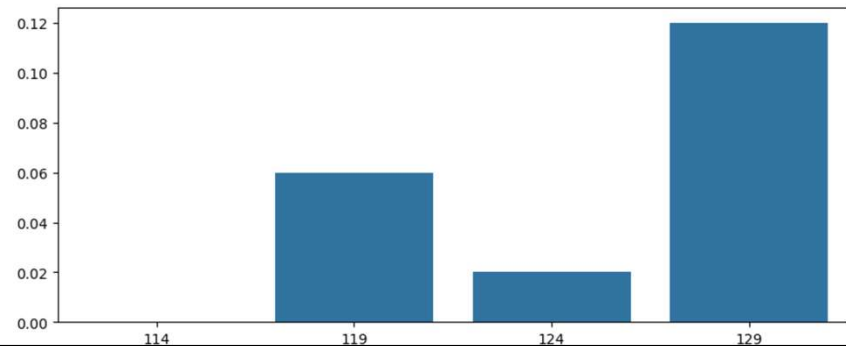
```
print(counts[0])
```

```
plt.figure(figsize=(10, 4))
```

```
sns.barplot(x=counts[1][:-1], y=counts[0]);
```

```
[114, 119, 124, 129, 134]
```

```
[0. 0.06 0.02 0.12]
```



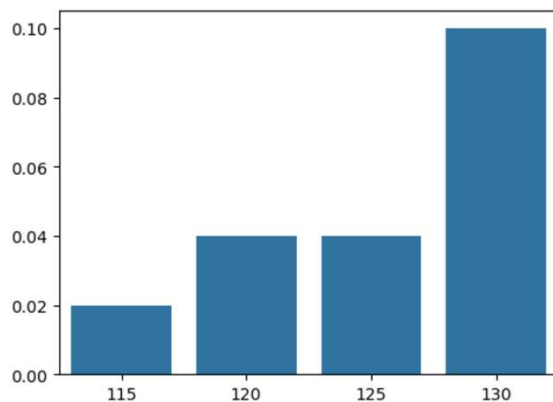
```
plt.figure(figsize=(12, 4))
```

```
bins_2 = range(115, 136, 5)
```

```
counts = np.histogram(data, bins_2, density=True)
```

```
plt.subplot(1, 2, 1)
```

```
sns.barplot(x=counts[1][:-1], y=counts[0])
```

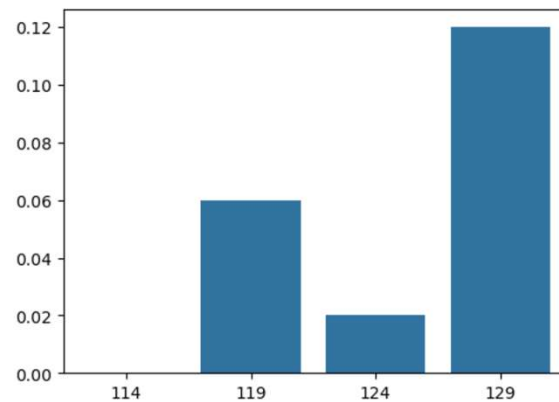


```
bins_3 = range(114, 136, 5)
```

```
counts = np.histogram(data, bins_3, density=True)
```

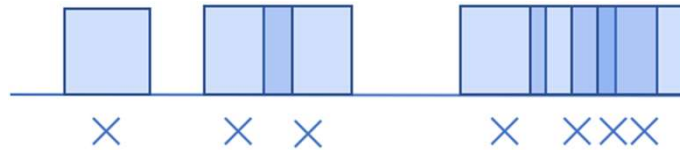
```
plt.subplot(1, 2, 2)
```

```
sns.barplot(x=counts[1][:-1], y=counts[0]);
```

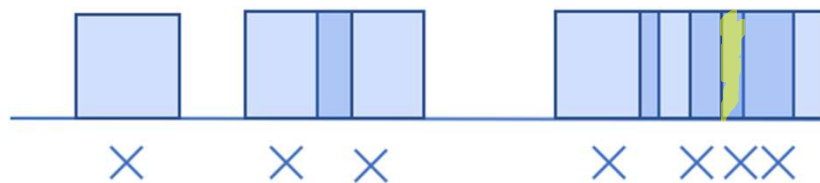


Estimating densities using kernels

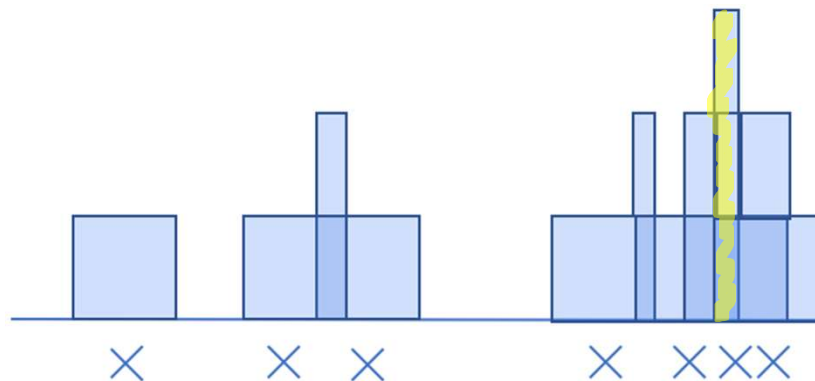
- How can we bypass the use of bin boundaries?
 - For instance, each data point could contribute **uniform** height to the histogram.

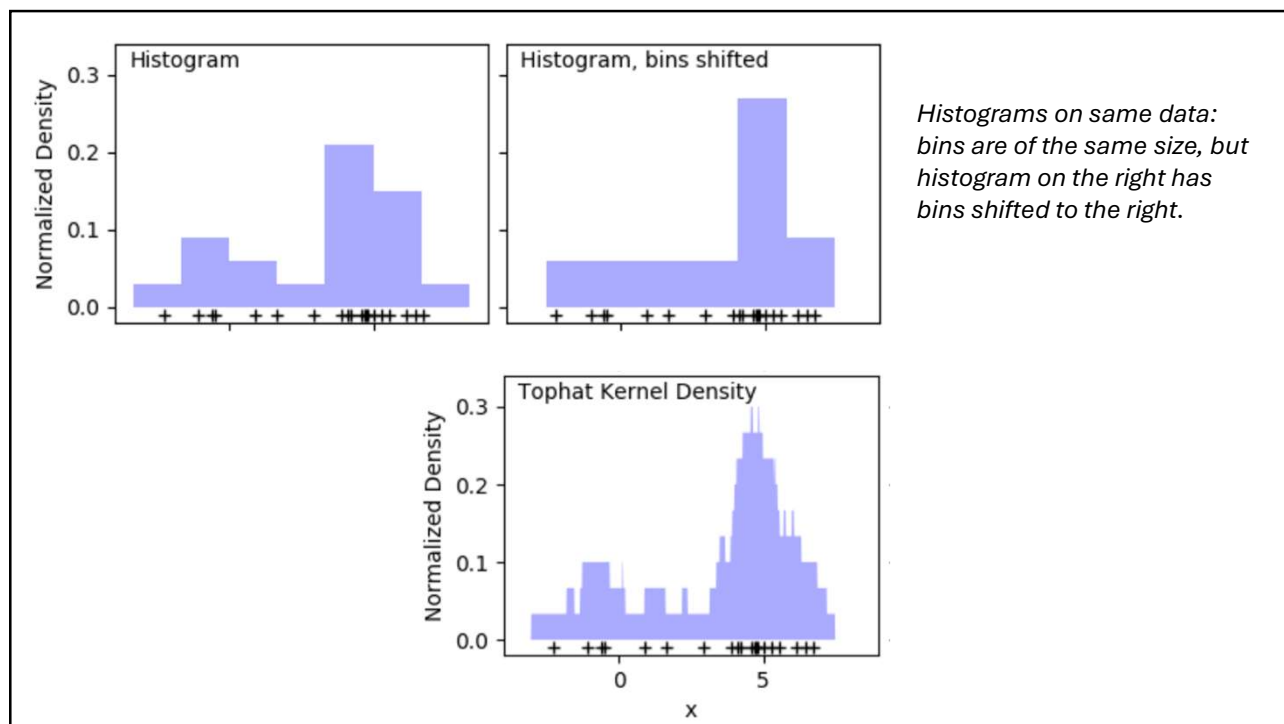


- For each position along the x-axis – including values with no data – we draw bars the heights of which equal to the sum of overlapping blocks at that position.



Each data point contributes a uniform height value. The shaded regions representing two or more overlapping blocks.



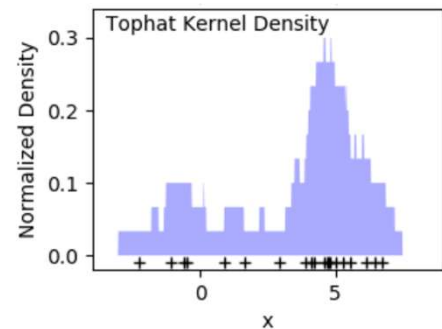


Estimating densities using kernels (cont'd.)

- Applying this approach to the problematic histogram produces a single, consistent visualization.
- Centering the box around each data point is a deterministic operation.
 - Not dependent on where we start our bin.
- We get an implied height for areas without a coverage.
 - That's good because samples are often unavailable for some areas of the sample space.
- Dense areas produce taller peaks.

Estimating densities using kernels (cont'd.)

- We can make inferences about the population based on a finite data sample.
- The square block we used is called a **kernel**, and the approach is called **Kernel Density Estimation** (KDE).
- Problem: leads to a more jagged histogram.
 - Can very close values lead to such drastic disagreement in density?

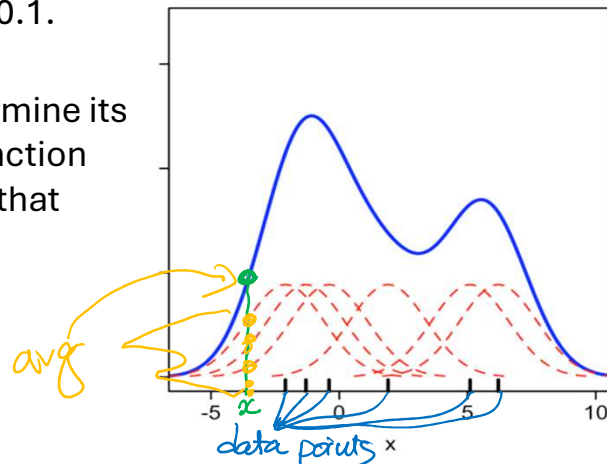


Using different kernels

- We are used to density curves being smooth (as opposed to the jagged plot with the square kernel).
- Instead of using a square block, what if we use a smoother curve as the kernel of each of the point.
 - Which curve should we try?

Using different kernels (cont'd.)

- We could, for instance, use a Gaussian kernel, centered at the data point and with a scale of 0.1.
- Rather than having each point contribute uniformly, we determine its contribution using a kernel function (e.g., a Gaussian) centered at that point.



Using different kernels (cont'd.)

- The density estimate at position x is obtained by summing the contributions of all kernel functions evaluated at x , with one kernel centered at each data point.
- For example, with a Gaussian kernel:

- Contribution of data point x' is: $\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-x')^2}{2\sigma^2}}$

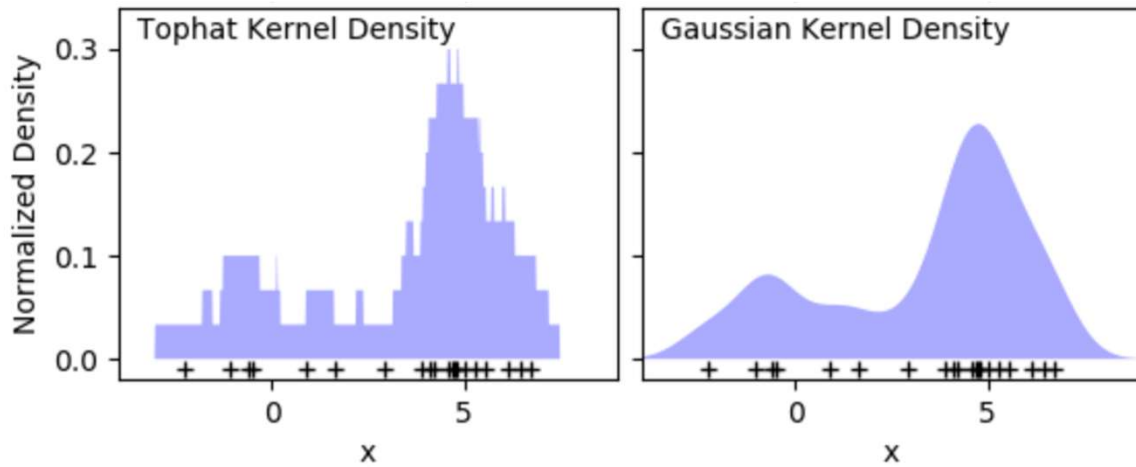
or in Python: `norm.pdf(x, x', sigma)`

- The resulting estimate is a smooth density function obtained by averaging these Gaussian kernels centered at each data point:

estimated density at position x

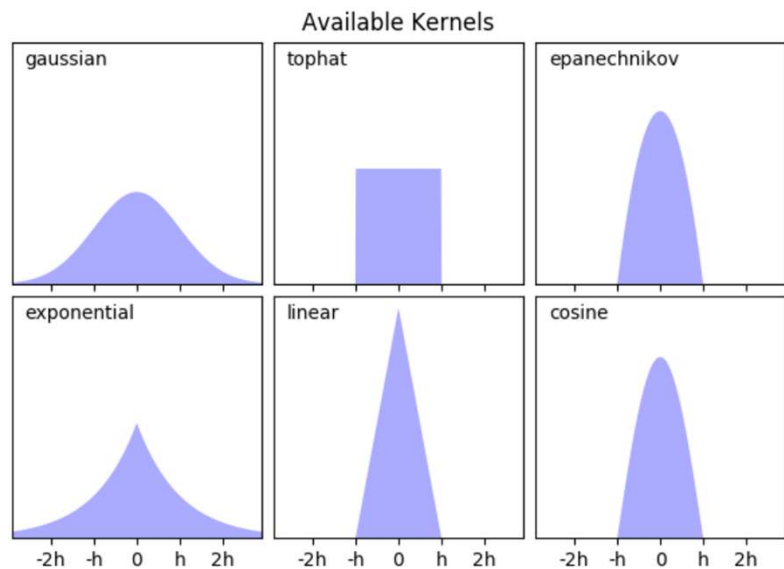
$$\hat{f}(x) = \frac{1}{|X|} \sum_{x' \in X} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-x')^2}{2\sigma^2}}$$

x: dataset



KDE kernels

- Other kernels are also available.



Differences between the kernels

- Each kernel emphasizes the contribution of neighboring points differently.
- KDE with some kernels is faster to compute than with others (e.g., Tophat versus Gaussian).
 - With a Tophat kernel of width k , the complexity is at most $O(n)$.
 - With the Gaussian kernel, the complexity is at most $O(n^2)$.

Using KDE on simulated data

- This example explores the KDE on data generated from a Gaussian with the following parameters:

$$X \sim \mathcal{N}(0, 0.5)$$

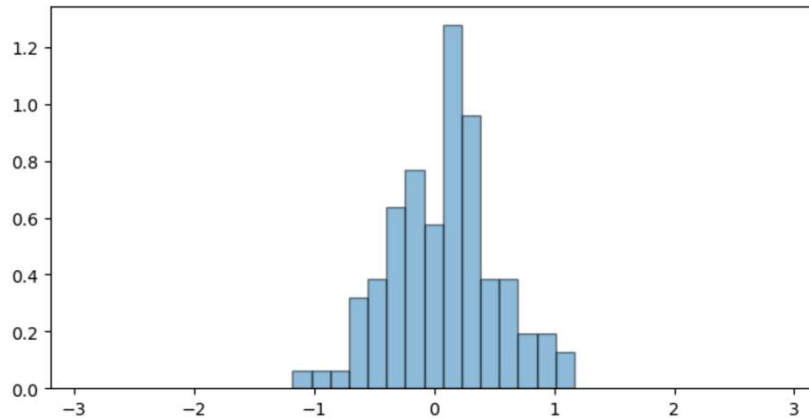
true distribution

1. Generate 100 points from the distribution above.
2. Build a histogram for the data.
3. Compare the KDE generated plot to the histogram.

```
x_mean, x_scale = 0, 0.5
x = np.random.normal(x_mean, x_scale, 100)
```

```
plt.figure(figsize=(8, 4))
```

```
plt.hist(x, edgecolor='k', alpha=0.5, bins=15, density=True)
plt.xlim(-3.2, 3.2)
plt.show()
```



Computing the KDE of a dataset

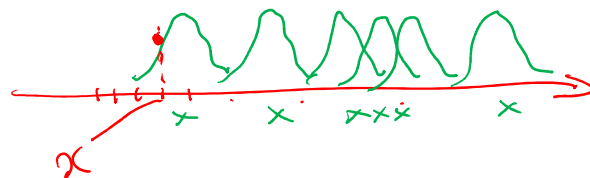
- Algorithm:

For each position x along the x-axis:

For each data point d in the dataset:

Compute contribution = $\text{Kernel}(x - d)$

Set $y(x)$ = average of all contributions



```
x_values = np.arange(-3, 3, 0.1)
```

```
# compute the KDE
```

```
kde = sp.stats.gaussian_kde(x, bw_method=0.8)
```

```
# estimate kernel density for the support (x-axis)
```

```
x_densities = kde.evaluate(x_values)
```

```
print(x[0:10])
```

```
print("*" * 75)
```

```
print(x_densities[0:10])
```

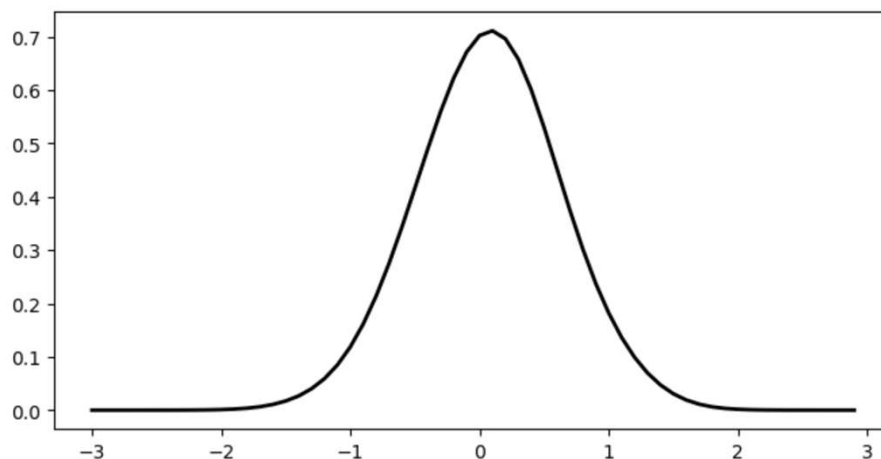
```
[ 0.61059585 -0.19727674 -0.39593168  0.28750031  0.04611475 -1.181126  
 0.43752158 -0.18514269  0.17716526  0.58887334]
```

```
*****  
[2.43763397e-08 9.90022183e-08 3.72106959e-07 1.29485377e-06  
 4.17391950e-06 1.24724956e-05 3.45830983e-05 8.90887760e-05  
 2.13575486e-04 4.77523479e-04]
```

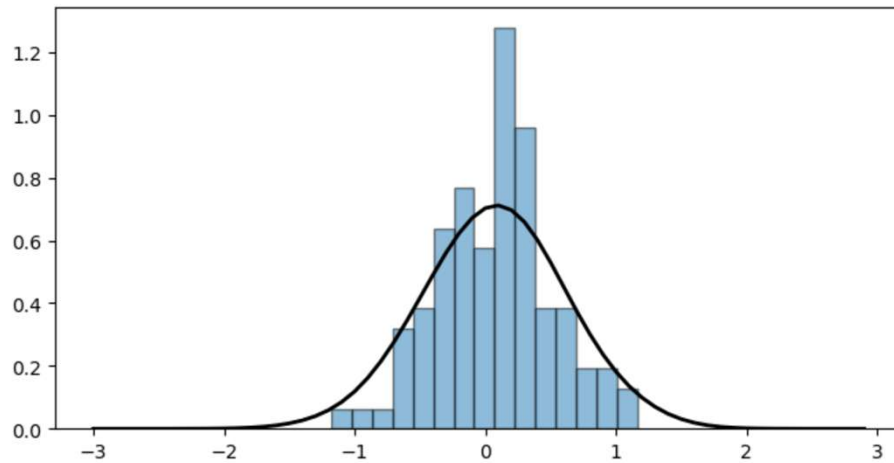
bandwidth.

returns density

```
plt.figure(figsize=(8, 4))  
plt.plot(x_values, x_densities, lw=2, color='black')  
plt.show()
```

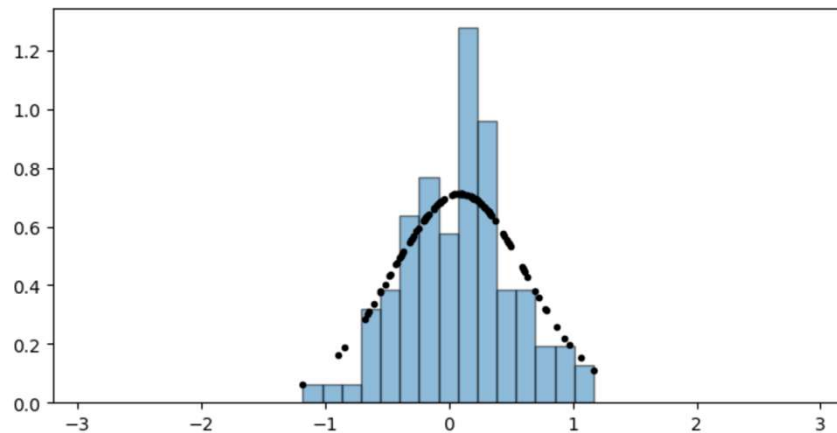


```
plt.figure(figsize=(8, 4))
plt.plot(x_values, x_densities, lw=2, color='black')
plt.hist(x, edgecolor='k', alpha=0.5, bins=15, density=True)
plt.show()
```



```
# estimate kernel density for the support (x data values)
x_densities = kde.evaluate(x)

plt.figure(figsize=(8, 4))
plt.plot(x, x_densities, '.', lw=2, color='black')
plt.hist(x, edgecolor='k', alpha=0.5, bins=15, density=True)
plt.xlim(-3.2, 3.2)
plt.show()
```



```

x_mean, x_scale = 0, 0.5
y_mean, y_scale = 9, 1

x_data = np.random.normal(x_mean, x_scale, 100)
y_data = np.random.normal(y_mean, y_scale, 100)
print("The first 10 samples from the first gaussian are:\n %s \n" % x_data[0:10])
print("The first 10 samples from the second gaussian are:\n %s \n" % y_data[0:10])

The first 10 samples from the first gaussian are:
[ 0.28654755  0.71677714  0.08934486 -0.91161396  0.40723638  0.70781508
 -0.1050297  -0.15188854 -0.75396335 -0.73763588]

The first 10 samples from the second gaussian are:
[10.25036688  9.55272474  9.05238717  9.57916111  9.38334195  9.47826107
 7.94731933  9.71146121  8.7356845  9.98217612]

all_data = np.concatenate([x_data, y_data]) # list_1 + list_2
print(len(all_data))

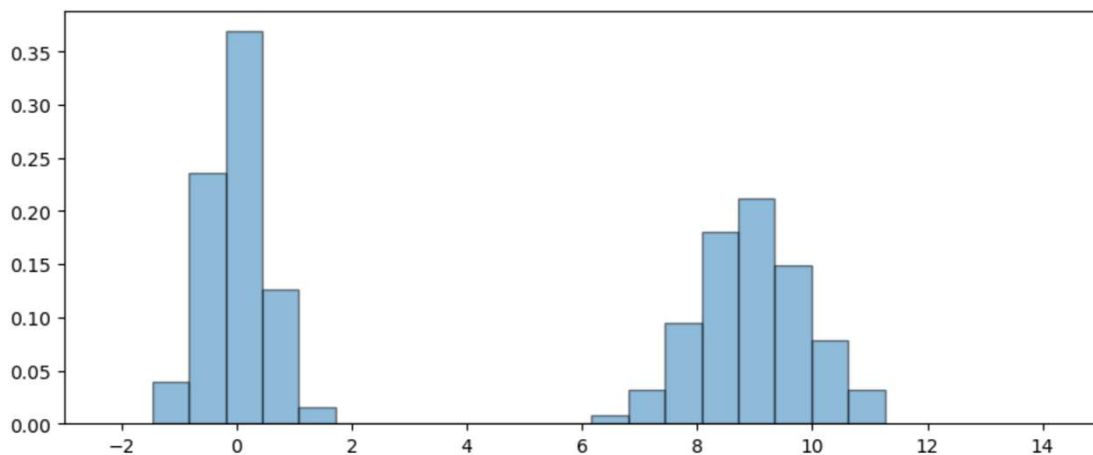
200

```

```

plt.figure(figsize=(10, 4))
plt.hist(all_data, bins=20, edgecolor="k", linewidth=1, density=True, alpha=0.5)
plt.xlim(-3, 15)
plt.show()

```



```

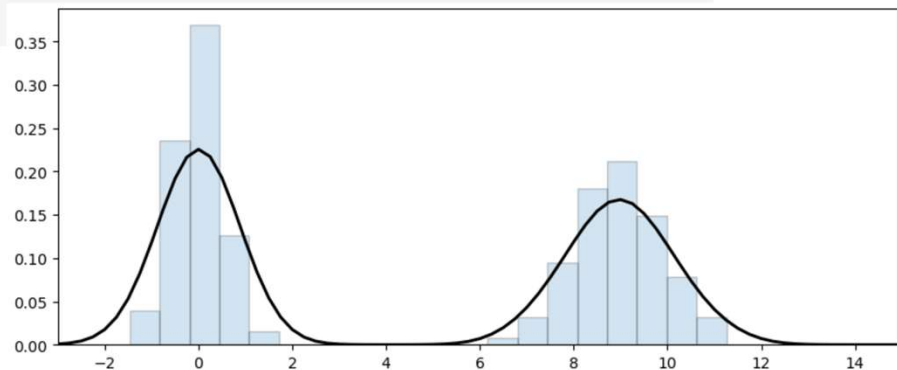
x_values = np.arange(-10, 20, 0.25)

# compute the KDE
kde = sp.stats.gaussian_kde(all_data, bw_method=0.16)

# estimate kernel density for the support (x-axis)
densities = kde.evaluate(x_values)

plt.figure(figsize=(10, 4))
plt.hist(all_data, bins=20, edgecolor="k", linewidth=1, density=True, alpha=0.2)
plt.plot(x_values, densities, lw=2, color='black')
plt.xlim(-3, 15)
plt.show()

```



Class participation questions

Post your answers to the questions below in Slack before the next class. Please prefix your post with: “KDE 2/25”

1. Is a probability distribution something we “discover” in data, or something we “assume”?
2. You measure the time users spend on your app during a session. You suspect the distribution is skewed with a long right tail. Why might KDE be a better first choice than assuming a normal distribution?